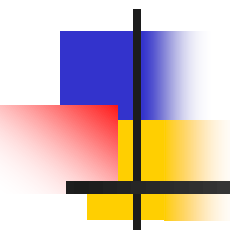
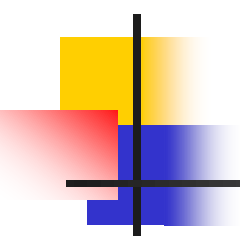


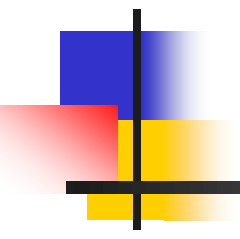
PART 2



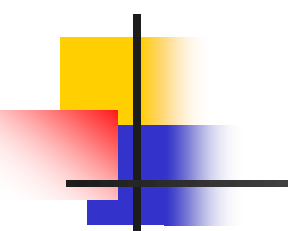
RELATIONAL DATABASES



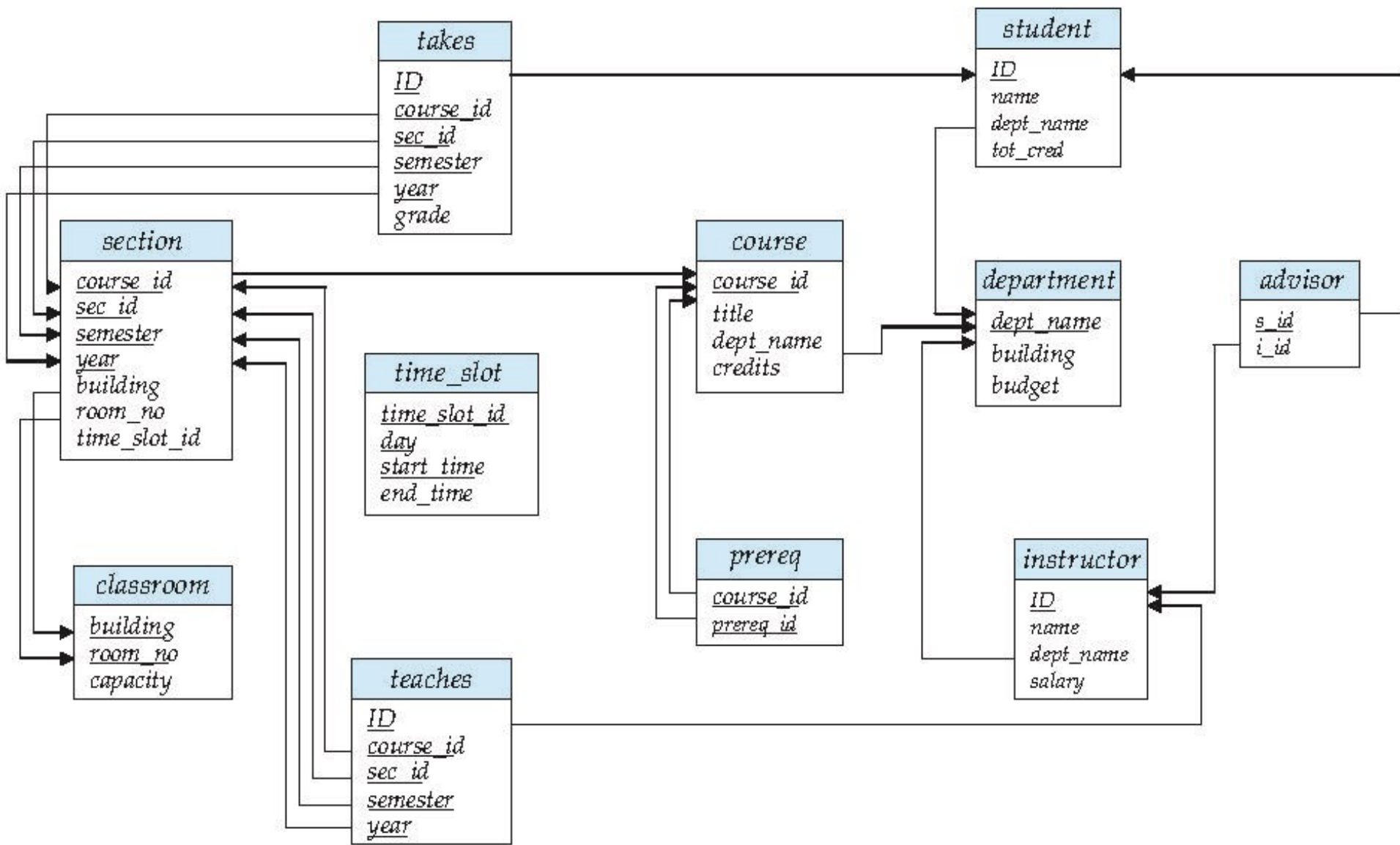
Chapter 4



Intermediate SQL

- 
- Join Expressions (连接表达式)
 - Views
 - Transactions
 - Integrity Constraints
 - SQL Data Types and Schemas
 - Authorization
 - Security in SQL

先讲4.5 扩展的数据类型



4.1 Joined Expressions

- **Join operations** take two relations and return as a result another relation.
- A *join* operation is a Cartesian product which requires that tuples in the two relations match (under some condition). It also specifies the attributes that are present in the result of the join
- The *join* operations are typically used as subquery expressions in the *from* clause

Joined Relations

- **Join condition** – defines which tuples in the two relations match, and what attributes are present in the result of the join.
- **Join type** – defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated.

<i>Join types</i>	<i>Join Conditions</i>
inner join left outer join right outer join full outer join	natural on <predicate> using ($A_1, A_1, \dots, A_n$)

Natural Join operations

For the following two relations

■ *course*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

Natural Join operations

- **select ***
from *course* **natural join** *prereq*
- **select ***
from *course* **join** *prereq* **on** *course.course_id=prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

Observe that, *course* **natural join** *prereq* **on** *course_id*
prereq information is missing for *CS-315* and
course information is missing for *CS-437*

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values.

Left Outer Join

course **natural left outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

■ **select ***

from *course* **natural left outer join** *prereq*

Right Outer Join

course **natural right outer join**

prereq

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

■ **select ***

from *course* **natural right outer join** *prereq*

Full Outer Join

course **natural full outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

■ **select ***

from *course* **natural full outer join** *prereq*

Joined Relations – Examples

- *course* **inner join** *prereq* on
course.course_id = prereq.course_id

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

What is the difference between the *inner join* mentioned above, and a *natural join*?

- *course* **left outer join** *prereq* on
course.course_id = prereq.course_id

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190
CS-315	Robotics	Comp. Sci.	3	<i>null</i>	<i>null</i>

Joined Relations – Examples

course **natural right outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

course **natural full outer join** *prereq* **using** (*course_id*)

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

4.2 Views

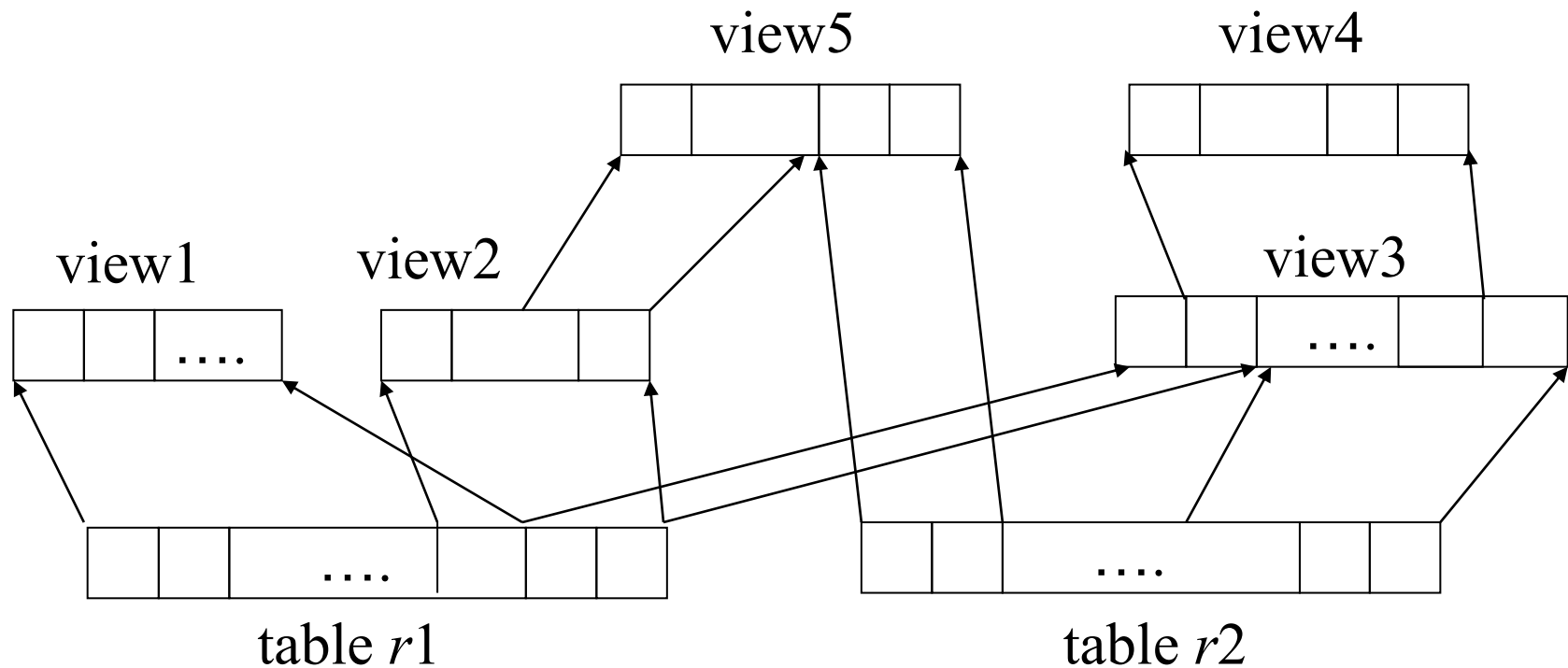
- In some cases, it is not desirable for all users to see the entire logical model (that is, all the actual relations stored in the database.)
- Consider a person who needs to know an instructors name and department, but not the salary. This person should see a relation described, in SQL, by

```
select ID, name, dept_name  
from instructor
```

- A **view** provides a mechanism to hide certain data from the view of certain users.
- Any relation that is not of the conceptual model but is made visible to a user as a “virtual relation” is called a **view**.

Views (cont.)

- E.g. Views definition



- Views = *projection* on *one or more relations*

Views (cont.)

- The view is also known as the *virtual relation/table*
 - only the definition of the view itself is stored in DBS, the *tuples* of the view is stored in the relations on which the view is defined
 - evaluation of a view is reduced to evaluation of the relation-algebra expression that define the view
- Note:
 - differences between the *with* clause and the *create view* clause
- Materialized views (实体化/物化视图)
 - both the definition and data of the views are stored in DBS

View Definition

- A view is defined using the **create view** statement which has the form

create view *v* **as** <query expression>

where <query expression> is any legal SQL expression.
The view name is represented by *v*.

- Once a view is defined, the view name can be used to refer to the *virtual relation* that the view generates.
- View definition is not the same as creating a new relation by evaluating the query expression
 - Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

Example Views

- A view of instructors without their salary
create view *faculty* as
 select *ID, name, dept_name*
 from *instructor*
- Find all instructors in the *Biology* department
 select *name*
 from *faculty*
 where *dept_name* = 'Biology'
- Create a view of department salary totals
 create view *departments_total_salary*(*dept_name*,
 total_salary) **as**
 select *dept_name*, sum (*salary*)
 from *instructor*
 group by *dept_name*;

Views Defined Using Other Views

- **create view** *physics_fall_2009* **as**
 select *course.course_id, sec_id, building, room_number*
 from *course, section*
 where *course.course_id = section.course_id*
 and *course.dept_name = 'Physics'*
 and *section.semester = 'Fall'*
 and *section.year = '2009';*
- **create view** *physics_fall_2009_watson* **as**
 select *course_id, room_number*
 from *physics_fall_2009*
 where *building= 'Watson';*

View Expansion

- Expand use of a view in a query/another view
create view *physics_fall_2009_watson* **as**
(select *course_id, room_number*
from (**select** *course.course_id, building, room_number*
from *course, section*
where *course.course_id = section.course_id*
and *course.dept_name = 'Physics'*
and *section.semester = 'Fall'*
and *section.year = '2009'*)
where *building = 'Watson'*;

Views Defined Using Other Views

- One view may be used in the expression defining another view
- A view relation v_1 is said to *depend directly* on a view relation v_2 if v_2 is used in the expression defining v_1
- A view relation v_1 is said to *depend on* view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- A view relation v is said to be *recursive* (递归) if it depends on itself.

View Expansion

- A way to define the meaning of views defined in terms of other views.
- Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations.
- View expansion of an expression repeats the following replacement step:
 - repeat**
 - Find any view relation v_i in e_1
 - Replace the view relation v_i by the expression defining v_i
 - until** no more view relations are present in e_1
- As long as the view definitions are not recursive, this loop will terminate

Update of a View

- Add a new tuple to *faculty* view which we defined earlier
insert into *faculty* values ('30765', 'Green',
'Music');

This insertion must be represented by the insertion of the tuple

('30765', 'Green', 'Music', null)

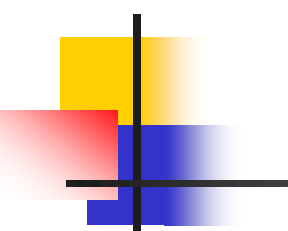
into the *instructor* relation

Some Updates cannot be Translated Uniquely

- **create view** *instructor_info* **as**
 select *ID, name, building*
 from *instructor, department*
 where *instructor.dept_name= department.dept_name;*
- **insert into** *instructor_info* **values** ('69987', 'White', 'Taylor');
 - which department, if multiple departments in Taylor?
 - what if no department is in Taylor?
- Most SQL implementations allow updates only on simple views
 - The **from** clause has only one database relation.
 - The **select** clause contains only attribute names of the relation, and does not have any expressions, aggregates, or **distinct** specification.
 - Any attribute not listed in the **select** clause can be set to null
 - The query does not have a **group** by or **having** clause.

And Some Not at All

- **create view** *history_instructors* **as**
 select *
 from *instructor*
 where *dept_name*= 'History';
- What happens if we insert ('25566', 'Brown', 'Biology', 100000)
into *history_instructors*?



Materialized Views

- **Materializing a view**: create a physical table containing all the tuples in the result of the query defining the view
- If relations used in the query are updated, the materialized view result becomes out of date
 - Need to **maintain** the view, by updating the view whenever the underlying relations are updated.

§ 4.4 Integrity Constraints

- Integrity constraints guard against accidental damage to the database, by ensuring that *authorized* changes to the database do not result in a loss of data consistency
 - An instructor name can not be *null*
 - No two instructors can have the same instructor *ID*
 - Every department name in the *course* relation must have a matching department name in the *department* relation
- Classification of integrity constraints and some examples
 - refer to Fig.4.0.1

	<i>attribute-level</i>	<i>tuple-level</i>	<i>relation-level</i>
static	■ data type	constraints	■ entity integrity
	■ data format, e.g. YY.MM.DD ■ domain constraints, e.g. 1 ■ null value, e.g. not null	among attributes values ➤ e.g. 2 ➤ mapping cardinality constraints	➤ e.g. 3 ■ referential integrity ■ functional dependency (§ 8) ■ statistical constraints ➤ e.g. 6
dynamics	constraints on updating of attribute values or attribute definition ➤ e.g. 4	constraints among attributes values ➤ e.g. 5	transaction constraint: atomy, consistency, isolation, durability (§ 14, 15, 16)

Some Examples of Integrity Constraints

- E.g.1 “the *salary of manager* should not be lower than \$1000” in *Employee*
- E.g.2 table $T(x, y, z)$, $z = x + y$, z is a ***derived*** attributes from x and y .
- E.g.3 “the *student#* for table *student* should not be null”
- E.g.4 “the *age of students* should only be added”
- E.g.5 when *employee tuples* is modified, $\text{new.sal} > \text{old.sal} + 0.5 * \text{age}$
- E.g.6 statistical constraints: in table *employee*, “the *salary of manager* should be four times more than that of workers”

4.4.1-4 Constraints on a Single Relation

- An SQL relation is defined using the **create table** command

create table *r* ($A_1 D_1, A_2 D_2, \dots, A_n D_n,$
 (integrity-constraint₁),
 ...,
 (integrity-constraint_k))

- *r* is the name of the relation
- each A_i is an attribute name in the schema of relation *r*
- D_i is the data type of values in the domain of attribute A_i

Constraints on a Single Relation (cont.)

- The allowed integrity constraints include
 - primary key
 - not null
 - unique
 - check (P), where P is a predicate

- **not null**
 - Declare *name* and *budget* to be **not null**
name **varchar(20) not null**
budget **numeric(12,2) not null**

Constraints on a Single Relation (cont.)

- The Unique Constraint is as follows

unique ($A_1, A_2, \dots, A_m$)

- it states that the attributes

$A_1, A_2, \dots, A_m$

form a **candidate key**

- **candidate keys are permitted to be null** (in contrast to primary keys)

- e.g. **create table** *customer*

(customer-id char(15)

customer-name char(15)

customer-city char(30),

primary key (*customer-id*)

unique (*customer-name*)

Constraints on a Single Relation (cont.)

- The check clause is applied to relation declaration as well as to domain declaration
 - **check** (P), where P is a predicate
 - e.g. ensure that semester is one of fall, winter, spring or summer:

```
create table section (  
    course_id      varchar (8),  
    sec_id         varchar (8),  
    semester       varchar (6),  
    year           numeric (4,0),  
    building       varchar (15),  
    room_number   varchar (7),  
    time slot id   varchar (4),  
    primary key   (course_id, sec_id, semester, year),  
    check         (semester in ('Fall', 'Winter', 'Spring', 'Summer'))  
);
```

4.4.5-6 Referential Integrity

- **Referential Integrity** (关联/参照完整性)
 - ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation
 - E.g. If “*Biology*” is a department name appearing in one of the tuples in the *instructor* relation, then there exists a tuple in the *department* relation for “*Biology*” (*next slide*)

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
69987	White	Math	<i>null</i>

instructor

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000
<i>null</i>	Painter	<i>null</i>

department

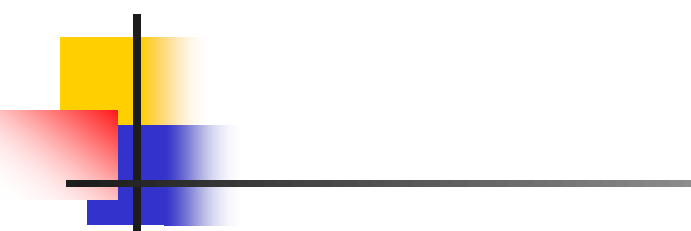
where is '**Math**' ?

Referential Integrity (cont.)

- Definition of *referential integrity/subset dependencies*
 - Let $r_1(R_1)$ and $r_2(R_2)$ be relations with primary keys K_1 and K_2 respectively, refer to Fig. 4.0.2
 - e.g. r_1 : *department*(dept_name, building, budgt)
 - r_2 : *course*(course_id, title, dept_name, credits),
- ▶
- the subset α of R_2 (e.g. *dept_name*) is a **foreign key** (from r_2 , e.g. *course*) referencing K_1 in relation r_1 (e.g. *dept_name* in *department*), if for every t_2 in r_2 there must be a tuple t_1 in r_1 such that

$$t_1[K_1] = t_2[\alpha]$$

- note: $\alpha = K_1 \subseteq R_2$



<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3

r_2

Figure A.5 The *course* relation.

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

r_1

Figure A.4 The *department* relation.

4.2.5 Referential Integrity (cont.)

- referential integrity constraint also called *subset dependency* since it can be written as

$$\Pi_{\alpha}(r_2) \subseteq \Pi_{K_1}(r_1)$$

- e.g. $\Pi_{dept_name}(course) \subseteq \Pi_{dept_name}(department)$

- a foreign key α of table r_2 references the primary key attributes K_1 of the *referenced table* r_1 

- A **good** DB design should ensures that any relation schema R_2 (and its any tuples) can only reference other relation schema R_1 through its foreign key

Referential Integrity (cont.)

- The primary key, candidate keys and foreign keys can be specified as parts of the SQL **create table** statement:
 - the **primary key** clause lists attributes that comprise the primary key.
 - the **unique key** clause lists attributes that comprise a candidate key.
 - the **foreign key** clause lists the attributes that comprise the foreign key and the name of the relation referenced by the foreign key.

By default, a foreign key references the primary key attributes of the referenced table.

Cascading Actions in Referential Integrity

- **create table** *course* (
 course_id **char**(5) **primary key**,
 title **varchar**(20),
 dept_name **varchar**(20) **references** *department*
)
- **create table** *course* (
 ...
 dept_name **varchar**(20),
 foreign key (*dept_name*) **references** *department*
 on delete cascade
 on update cascade,
 ...
)
- alternative actions to cascade: **set null, set default**

Cascading Actions in Referential Integrity

- The *delete/update* operations on the *referenced* relation *department* will result in the *delete/update* on the *referencing* relation *course*

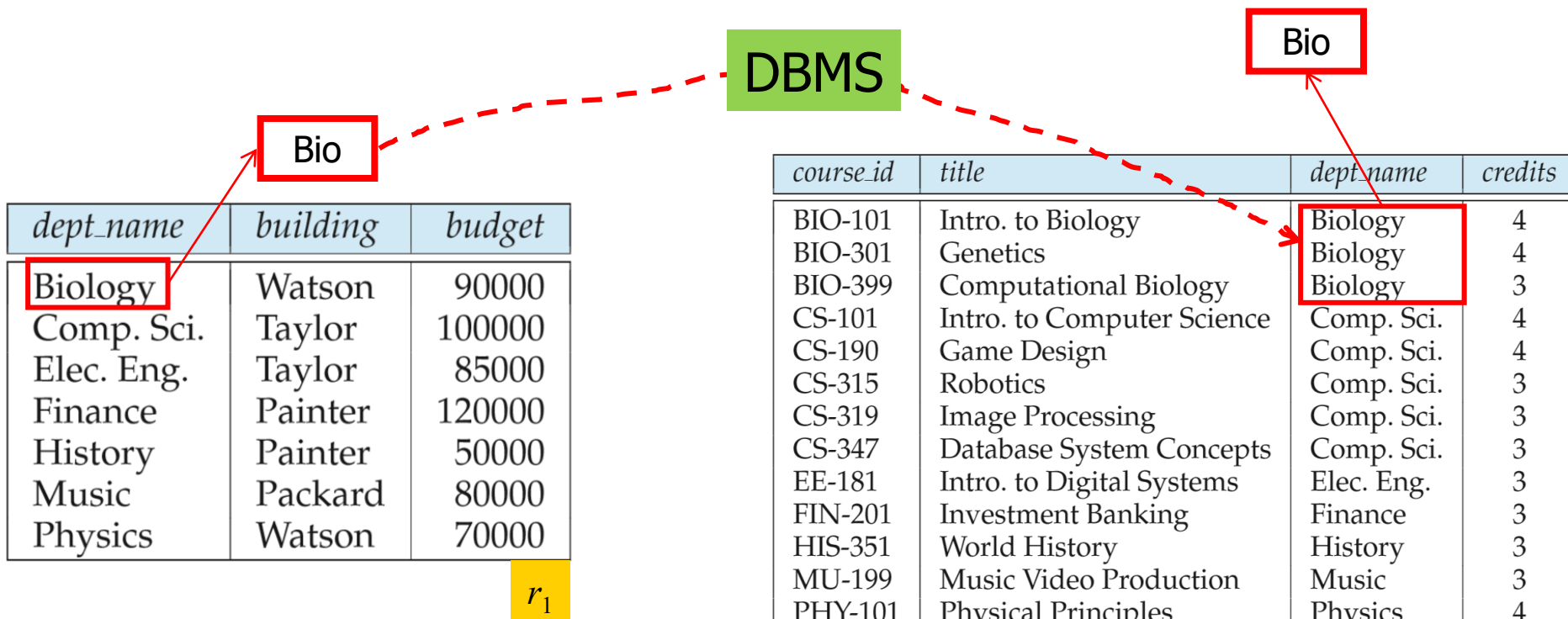


Figure A.4 The *department* relation.

Figure A.5 The *course* relation.

Integrity Constraint Violation During Transactions

■ E.g.

```
create table person (  
  ID char(10),  
  name char(40),  
  mother char(10),  
  father char(10),  
  primary key ID,  
  foreign key father references person,  
  foreign key mother references person)
```

- How to insert a tuple without causing constraint violation ?
 - insert father and mother of a person before inserting person
 - OR, set father and mother to null initially, update after inserting all persons (not possible if father and mother attributes declared to be **not null**)
 - OR defer constraint checking (next slide)

实验要求

- 以SQL Server为平台，定义2张表间的无级联modification、有级联modification的外键关联，
判断：当增/删/改一张表中的主键、外键值时，DBMS的反应

（知乎话题）

数据库系统设计时，是否使用外键？！

- 外键反映了不同数据项间的逻辑关联和约束关系，是一种客观存在
 - e.g. *dept_name* in the table *department*, *course*
- 对外键的处理方式反映了对这种数据相互间关系的处理态度，需要根据实际情况而定，考虑数据完整性、处理效率等多种因素
 - 并非简单的“需要外键”，“不设外键”（e.g. 部分MySQL课程、MySQL社区的论点）

（知乎话题）

数据库系统设计时是否使用外键？！

- 方式1：设置数据库表的外键

- 优点：

- DBMS自动维护数据间的关联、约束关系

- 缺点：

- 数据导入、增删改时，多个表间的相互关联，保证了数据一致性、完整性，但影响到整体数据处理效率（额外的DBMS处理开销）

- e.g.插入一条`dept_name`不存在的`course`元组，失败，并影响前面已经成功插入的数据——局部错误影响整体

（知乎话题）

数据库系统设计时是否使用外键？！

- **方式2：**不设置外键，由应用程序在业务逻辑层维护数据一致性、完整性
 - 优点
 - 1) 避免局部错误影响整体进度；
 - 2) 业务逻辑层的数据一致性处理的速度可以比较快，e.g. 将数据提至内存中批量处理
 - **缺点：对应用层业务处理程序设计的额外要求**
- 上述2种方式用在对数据一致性完整性要求高的场景下，e.g. 银行
- **方式3：**不设置外键，业务层应用逻辑也不处理数据间一致性
 - 牺牲数据的正确性，换取数据处理的效率
 - e.g. 大数据应用场景下，对数据精度、正确性要求不高

4.4.7 Complex Check Conditions and Assertions

- In the SQL standard, the some construct such as *check* and *assertion* are defined to specify the complex integrity constraints.
 - **However, they not currently supported by most database systems (?)**
- E.g. For the relation *section*, the time_slot_id in each tuple should be is actually the identifier of a time_slot in the *time_slot* relation. This constraint can be defined as

check (*time_slot_id* in (select *time_slot_id* from *time_slot*))

- *check* in create table

```
create table student (  
    ID          varchar(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    tot_cred    numeric(3,0),  
    primary key (ID),  
    foreign key (dept_name) references department)  
    check(tot_cred>0)
```


Complex Check Conditions and Assertions (cont.)

- An **assertion** is a predicate expressing a condition that we wish the database always to satisfy
 - e.g. domain constraints, referential-integrity constraint
- An assertion in SQL takes the form
create assertion <assertion-name> **check** <predicate>
 - **Also not supported by anyone, e.g. SQL Server等数据库系统似乎不支持创建assertion !!!**
- **Why ?**

When an assertion is made, the DBMS tests it for validity. Any modification to DB is allowed only if it does not cause that assertion to be violated

 - this testing may introduce a significant amount of overhead, hence **assertions should be used with great care**

Complex Check Conditions and Assertions (cont.)

- For each tuple in the *student* relation, the value of the attribute *tot_cred* must equal the sum of credits of courses that the student has completed successfully
- **create assertion** *credits_earned_constraints* **check**
(**not exists** (select *ID*
 from *student*
 where *tot_cred*
 <> select sum(*credits*)
 from *takes* **natural join** *course*
 where *student.ID=takes.ID*
 and *grade* **is not null** **and** *grade* <> 'F')
)

§ 4.5 SQL Data Types and Schemas

4.5.1 Data and Time Types in SQL

- In addition to the basic data types, e.g. char(n), varchar(n), int, smallint, numeric(p, d), real, double precision, float(n), the other built-in data types include
 - date: Dates, containing a (4 digit) year, month and date
 - e.g. date '2017-7-27'
 - time: Time of day, in hours, minutes and seconds
 - e.g. time '09:00:30.75'
 - timestamp: date plus time of day
 - e.g. timestamp '2017-7-27 09:00:30.75'
 - interval: period of time
 - if x and y are of type date, then $x-y$ is an interval whose value is the number of days from date x to date y
 - e.g. interval '1' day = '2017-7-27' - '2017-7-28'

Built-in Data Types in SQL (cont.)

- cast *e* as *t*
 - convert a character string (or string valued expression) *e* to the type *t*, e.g. date/time/timestamp
 - e.g. **cast** '2017-07-20' as **date**
 - e.g. **cast** '09:00:30.75' as **time**
- For a *day* or *time* value *d*, its individual fields can be extracted
 - e.g. **extract** (*year from* 2017-07-20) =2017
 - e.g. **extract** (*timezone_hour from* 2017-7-27 09:00:30.75) =09

4.5.4 Large-Object Types

- Large objects (photos, videos, CAD files, etc.) are stored as a *large object*
- **blob**: binary large object (二进制大型对象)
 - the object is a large collection of uninterpreted binary data, whose interpretation is left to an application outside of the database system
- **clob**: character large object
 - the object is a large collection of character data
- When a query returns a large object, *a pointer* is returned rather than the large object itself !

Large-Object Types (cont.)

- E.g.

book_view **clob**(10KB)

image **blob**(10MB)

movie **blob**(2GB)

- Now, Oracle, DB2, and SQL Server also support the XML data type integrated in relational schemas, i.e. the attributes of XML data type in relational tables

e.g. Create table *book*

book_id **int primary key**

book_view **clob**(10KB)

image **blob**(10MB)

movie **blob**(2GB)

catalog XML

4.5.5 User-defined Types

- The **create type** clause can be used to define new user-defined types
 - e.g. **create type** *Dollars* **as numeric (12,2) final**
create table *department*
 (*dept_name* **varchar** (20),
 building **varchar** (15),
 budget ***Dollars***);
- **cast**, i.e. *convert*, values of one type to another domain
 - **cast** (*department.budgt* to *numeric(12.2)*)
 , then the expression
 (***department.budgt*** + 20)
 can be evaluated

Domains

- **create domain** construct in SQL-92 creates user-defined domain types

create domain *person_name* **char(20) not null**

- Types and domains are similar. **Domains can have constraints**, such as **not null**, specified on them.
- **create domain** *degree_level* **varchar(10)**
constraint *degree_level_test*
check (value in ('Bachelors', 'Masters', 'Doctorate'));

SQL Server2005 类型转换

目标数据类型:	源数据类型:																									
	binary	varbinary	char	varchar	nchar	nvarchar	datetime	smalldatetime	decimal	numeric	float	real	bigint	int(INT4)	smallint(INT2)	tinyint(INT1)	money	smallmoney	bit	timestamp	uniqueidentifier	image	ntext	text	sql_variant	xml
binary																										
varbinary																										
char																										
varchar																										
nchar																										
nvarchar																										
datetime																										
smalldatetime																										
decimal																										
numeric																										
float																										
real																										
bigint																										
int(INT4)																										
smallint(INT2)																										
tinyint(INT1)																										
money																										
smallmoney																										
bit																										
timestamp																										
uniqueidentifier																										
image																										
ntext																										
text																										
sql_variant																										
xml																										

- 显式转换
- 隐式转换
- 不允许转换
- * 要求使用显式转换，以防止使用隐式转换时降低了精度或减少了小数位数
- 仅当源或目标为非类型化的 xml 时，才支持 XML 数据类型之间的隐式转换；否则，它们之间的转换必须为显式转换。

CAST 和 CONVERT (Transact-SQL)

 全部折叠  语言筛选器: 全部

将一种数据类型的表达式显式转换为另一种数据类型的表达式。`CAST` 和 `CONVERT` 提供相似的功能。

 [Transact-SQL 语法规则](#)

语法

Syntax for `CAST`:

```
CAST ( expression AS data_type [ (length) ] )
```

Syntax for `CONVERT`:

```
CONVERT ( data_type [ (length) ] , expression [ , style ] )
```

备注

隐式转换指那些没有指定 `CAST` 或 `CONVERT` 函数的转换。显式转换指那些需要指定 `CAST` 或 `CONVERT` 函数的转换。以下图例显示了可对 SQL Server 2005 系统提供数据类型转换。其中包括 `xml`、`bigint` 和 `sql_variant`。不存在对 `sql_variant` 数据类型的赋值进行的隐式转换，但是存在转换为 `sql_variant` 的隐式转换。

A. 同时使用 CAST 和 CONVERT

每个示例都检索列表价格的第一位是 3 的产品的名称，并将 ListPrice 转换为 int。

```
-- Use CAST
USE AdventureWorks;
GO
SELECT SUBSTRING(Name, 1, 30) AS ProductName, ListPrice
FROM Production.Product
WHERE CAST(ListPrice AS int) LIKE '3%';
GO

-- Use CONVERT.
USE AdventureWorks;
GO
SELECT SUBSTRING(Name, 1, 30) AS ProductName, ListPrice
FROM Production.Product
WHERE CONVERT(int, ListPrice) LIKE '3%';
GO
```

varchar, e.g. 350.00

B. 使用包含算术运算符的 CAST

以下示例将本年度截止到现在的全部销售额 (SalesYTD) 除以佣金百分比 (CommissionPCT)，从而得出单列计算结果 (Computed)。在舍入到最接近的整数后，将此结果转换为 int 数据类型。

```
USE AdventureWorks;
GO
SELECT CAST(ROUND(SalesYTD/CommissionPCT, 0) AS int) AS 'Computed'
FROM Sales.SalesPerson
WHERE CommissionPCT != 0;
GO
```

C. 使用 CAST 进行连接

以下示例使用 CAST 连接非字符型非二进制表达式。

```
USE AdventureWorks;
GO
SELECT 'The list price is ' + CAST(ListPrice AS varchar(12)) AS ListPrice
FROM Production.Product
WHERE ListPrice BETWEEN 350.00 AND 400.00;
GO
```

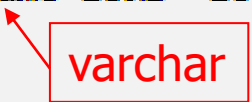


float

D. 使用 CAST 生成可读性更高的文本

以下示例使用选择列表中的 CAST 将 Name 列转换为 char(10) 列。

```
USE AdventureWorks;
GO
SELECT DISTINCT CAST(p.Name AS char(10)) AS Name, s.UnitPrice
FROM Sales.SalesOrderDetail s JOIN Production.Product p on s.ProductID = p.ProductID
WHERE Name LIKE 'Long-Sleeve Logo Jersey, M';
GO
```



varchar

4.5.2 Default Values

- **create table** *student*
 (*ID* **varchar** (5),
 name **varchar** (20) **not null**,
 dept_name **varchar** (20),
 tot_cred **numeric** (3,0) **default** 0,
 primary key (*ID*))
- **insert into** *student*(*ID*, *name*, *dept_name*)
 values ('12789', 'Newman', 'Comp.Sci')

4.5.3 Index Creation

- **create table** *student*
 (*ID* **varchar** (5),
 name **varchar** (20) **not null**,
 dept_name **varchar** (20),
 tot_cred **numeric** (3,0) **default** 0,
 primary key (*ID*))
- **create index** *studentID_index* **on** *student*(*ID*)
- Indices are data structures used to speed up access to records with specified values for index attributes
 - e.g. **select** *
 from *student*
 where *ID* = '12345'

can be executed by using the index to find the required record, without looking at all records of *student*

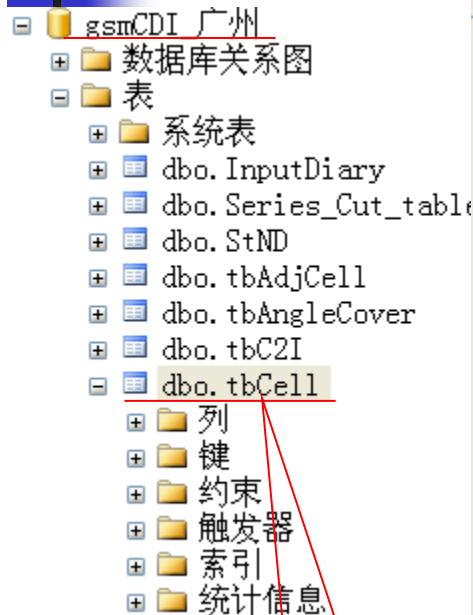
More on indices in Chapter 11

4.5.5 Schemas, Catalogs, and Environments

- How to name the relations in DB, or how to specify the names of the relations in DB
- The three-level hierarchy for naming relations
 - *catalogs* (全文目录), each of which can contain *schemas*;
The *catalog* is also called *database*
 - *schemas* (架构), in which the relations and views are stored
the **Schema** object represents an ownership context for a Microsoft SQL Server database object.
 - relations and views
e.g. a three-part name for the relation
catalog5.bank_schema.account



Schemas, Catalogs, and Environments



右键属性

脚本 ▾ 帮助	
存储 分区方案 数据空间 索引空间 文本文件组 文件组 行计数 已对表进行分区 当前连接参数 服务器 数据库 用户 复制 对表进行复制 说明 Schema 创建日期 名称 系统对象 选项 ANSI NULLs 带引号的标识符	
23.539 MB	1.148 MB
PRIMARY	33629
False	
YEWEN	gsmCDI_广州
YEWEN\yewenbupt	
False	
dbo	2010-4-29 20:53
tbCell	False
True	True

- SQL environments, including
 - connection, catalog, schema, user identifier

§ 4.6 Authorization

- Forms of authorization on parts of the *database* include
 - **Read** - allows reading, but not modification of data
 - **Insert** - allows insertion of new data, but not modification of existing data
 - **Update** - allows modification, but not deletion of data
 - **Delete** - allows deletion of data
- Forms of authorization to modify the *database schema* (covered in Chapter 8)
 - **resources** - allows creation of new relations
 - **alteration** - allows addition or deletion of attributes in a relation
 - **drop** - allows deletion of relations
 - **index**

Authorization Specification in SQL

- The **grant** statement is used to confer authorization
 - grant** <privilege list>
on <relation name or view name> **to** <user list>
<user list> is:
 - a user-id
 - **public**, which allows all valid users the privilege granted
 - a role (more on this in Chapter 8)
- Granting a privilege on a view does not imply granting any privileges on the underlying relations.
- The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator).

Privileges in SQL

- **select**: allows read access to relation, or the ability to query using the view

- Example: grant users U_1 , U_2 , and U_3 **select** authorization on the *instructor* relation:

grant select on *instructor* to U_1 , U_2 , U_3

- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.
- **all privileges**: used as a short form for all the allowable privileges

Revoking Authorization in SQL

- The **revoke** statement is used to revoke authorization.

revoke <privilege list>

on <relation name or view name> **from** <user list>

- Example:

revoke select on *branch* **from** U_1, U_2, U_3

- <privilege-list> may be **all** to revoke all privileges the revoker may hold.
- If <revokee-list> includes **public**, all users lose the privilege except those granted it explicitly.

Roles

- **create role** instructor;
- **grant** *instructor* **to** Amit;
- Privileges can be granted to roles:
 - **grant select on** *takes* **to** *instructor*;
- Roles can be granted to users, as well as to other roles
 - **create role** *teaching_assistant*
 - **grant** *teaching_assistant* **to** *instructor*;
 - *Instructor* inherits all privileges of *teaching_assistant*
- Chain of roles
 - **create role** *dean*;
 - **grant** *instructor* **to** *dean*;
 - **grant** *dean* **to** Satoshi;

Authorization on Views

- **create view** *geo_instructor* **as**
 (select *
 from *instructor*
 where *dept_name* = 'Geology');
- **grant select on** *geo_instructor* **to** *geo_staff*
- Suppose that a *geo_staff* member issues
 - **select** *
 from *geo_instructor*;
- What if
 - *geo_staff* does not have permissions on *instructor*?
 - creator of view did not have some permissions on *instructor*?

Other Authorization Features

- **references** privilege to create foreign key
 - **grant reference** (*dept_name*) **on** *department* **to** Mariano;
 - why is this required?
- **transfer of privileges**
 - **grant select on** *department* **to** Amit **with grant option**;
 - **revoke select on** *department* **from** Amit, Satoshi **cascade**;
 - **revoke select on** *department* **from** Amit, Satoshi **restrict**;
- Etc. read Section 4.6 for more details we have omitted here.

Appendix A *create table* in T-SQL

```
CREATE TABLE
    [ database_name . [ schema_name ] . | schema_name . ] table_name
    ( { <column_definition> | <computed_column_definition> }
      [ <table_constraint> ] [ ,...n ] )
    [ ON { partition_scheme_name ( partition_column_name ) | filegroup
      | "default" } ]
    [ { TEXTIMAGE_ON { filegroup | "default" } } ]
    [ ; ]
```

分区

文件组

物理设计

```
<column_definition> ::=
column_name <data_type>
    [ COLLATE collation_name ]
    [ NULL | NOT NULL ]
    [
        [ CONSTRAINT constraint_name ] DEFAULT constant_expression ]
    | [ IDENTITY [ ( seed ,increment ) ] [ NOT FOR REPLICATION ]
    ]
    [ ROWGUIDCOL ] [ <column_constraint> [ ...n ] ]
```



```

<column_constraint> ::=
[ CONSTRAINT constraint_name ]
{
    { PRIMARY KEY | UNIQUE }
    [ CLUSTERED | NONCLUSTERED ]
    [
        WITH FILLFACTOR = fillfactor
        | WITH ( < index_option > [ , ...n ] )
    ]
}

```

聚集/非聚集索引
(物理设计)

外键

```

]
[ ON { partition_scheme_name ( partition_column_name )
    | filegroup | "default" } ]
[ FOREIGN KEY ]
REFERENCES [ schema_name . ] referenced_table_name [ ( ref_column ) ]
[ ON DELETE { NO ACTION | CASCADE | SET NULL | SET DEFAULT } ]
[ ON UPDATE { NO ACTION | CASCADE | SET NULL | SET DEFAULT } ]
[ NOT FOR REPLICATION ]
| CHECK [ NOT FOR REPLICATION ] ( logical_expression )
}

```

```

<computed_column_definition> ::=
column_name AS computed_column_expression
[ PERSISTED [ NOT NULL ] ]
[
  [ CONSTRAINT constraint_name ]
  { PRIMARY KEY | UNIQUE }
  [ CLUSTERED | NONCLUSTERED ]
  [
    WITH FILLFACTOR = fillfactor
    | WITH ( <index_option> [ , ...n ] )
  ]
  | [ FOREIGN KEY ]
  REFERENCES referenced_table_name [ ( ref_column ) ]
  [ ON DELETE { NO ACTION | CASCADE } ]
  [ ON UPDATE { NO ACTION } ]
  [ NOT FOR REPLICATION ]
  | CHECK [ NOT FOR REPLICATION ] ( logical_expression )
  [ ON { partition_scheme_name ( partition_column_name )
    | filegroup | "default" } ]
]
  
```

```
< table_constraint > ::=
[ CONSTRAINT constraint_name ]
{
    { PRIMARY KEY | UNIQUE }
      [ CLUSTERED | NONCLUSTERED ]

      (column [ ASC | DESC ] [ ,...n ] )

    [
      WITH FILLFACTOR = fillfactor
    |WITH ( <index_option> [ , ...n ] )
    ]
    [ ON { partition_scheme_name (partition_column_name)
      | filegroup | "default" } ]
    | FOREIGN KEY
      ( column [ ,...n ] )
      REFERENCES referenced_table_name [ ( ref_column [ ,...n ] ) ]
      [ ON DELETE { NO ACTION | CASCADE | SET NULL | SET DEFAULT } ]
      [ ON UPDATE { NO ACTION | CASCADE | SET NULL | SET DEFAULT } ]
      [ NOT FOR REPLICATION ]
    | CHECK [ NOT FOR REPLICATION ] ( logical_expression )
}
```

```
<index_option> ::=  
{  
    PAD_INDEX = { ON | OFF }  
    | FILLFACTOR = fillfactor  
    | IGNORE_DUP_KEY = { ON | OFF }  
    | STATISTICS_NORECOMPUTE = { ON | OFF }  
    | ALLOW_ROW_LOCKS = { ON | OFF }  
    | ALLOW_PAGE_LOCKS = { ON | OFF }  
}
```

备注

在 SQL Server 2005 中，每个数据库最多可包含 20 亿个表，每个表可包含 1,024 列。表的行数及总大小仅受可用存储空间的限制。每行最多包括 8,060 个字节。对于带 `varchar`、`nvarchar`、`varbinary` 或 `sql_variant` 列（导致已定义表的总宽超过 8,060 字节）的表，此限制将放宽。其中每列的长度仍必须在 8,000 字节的限制内，但是它们的总宽可能超过表的 8,060 字节的限制。有关详细信息，请参阅[行溢出数据超过 8 KB](#)。

每个表最多可以有 249 个非聚集索引和 1 个聚集索引。其中包括为支持表中所定义的 `PRIMARY KEY` 和 `UNIQUE` 约束而生成的索引。

通常情况下，为表和索引分配空间时，每次以一个区为增量单位。当创建表或索引时，首先从混合区为其分配页，直到它具有足够的页填满一个统一区。当足够的页填满统一区后，每当当前分配的区填满时，将再为其分配另一个区。若要获得关于由表分配和占用的空间量的报表，请执行 `sp_spaceused`。

数据库引擎 在列定义中并不强制以特定的顺序指定 `DEFAULT`、`IDENTITY`、`ROWGUIDCOL` 或列约束。

创建表后，即使 `QUOTED IDENTIFIER` 选项在创建表时设置为 `OFF`，该选项在表的元数据中仍存储为 `ON`。

临时表

可以创建本地临时表和全局临时表。本地临时表仅在当前会话中可见，而全局临时表在所有会话中都可见。临时表不能分区。

本地临时表的名称前面有一个数字符号 (`#table_name`)，而全局临时表的名称前面有两个数字符号 (`##table_name`)。

SQL 语句使用 `CREATE TABLE` 语句中为 `table_name` 指定的值引用临时表，例如：

Appendix B 实际系统中的外键定义

- DBMS不会自动生成外键，需要用户显示定义外键

- e.g.1

r_1 : *department*(dept_name, building, budgt)

r_2 : *course*(course_id, title, **dept_name**, credits),

- e.g.2

branch(branch-name, branch-city, assets),

account(account-number, branch-name, balance)

- 如何在已经创建的关系表中，加入外键定义？？

- *create assertion* ? ——*SQL Server*不支持

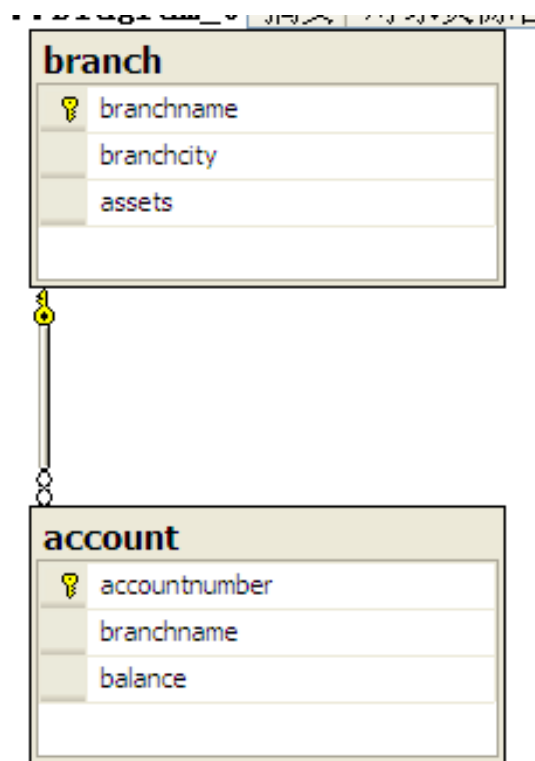
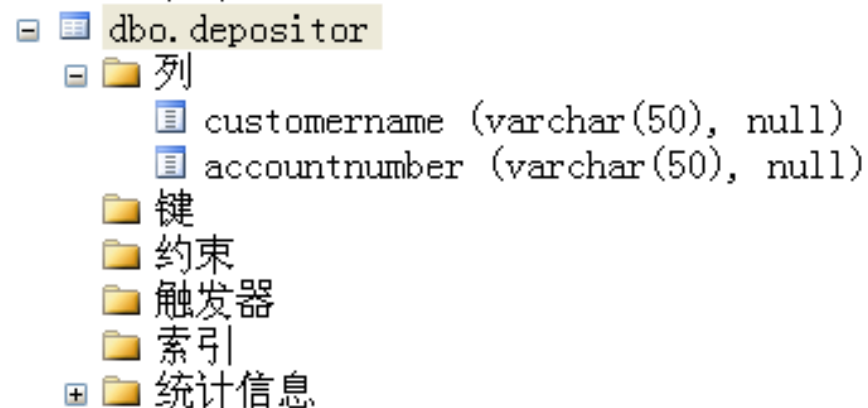
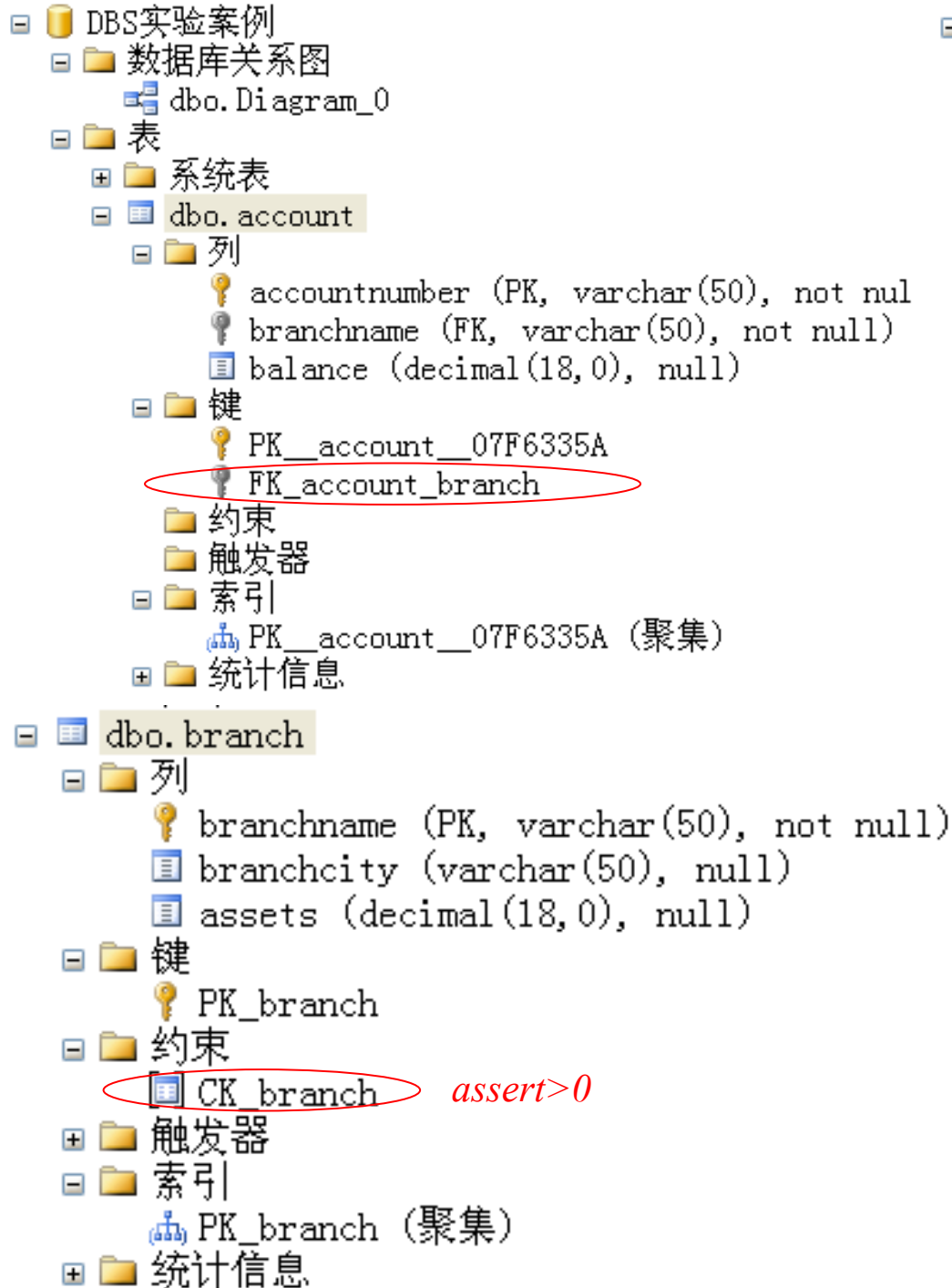
- *Alter*

```
USE [DBS实验案例]
GO
/***** 对象: Table [dbo].[account]    脚本日期: 03/20/2014 12:44:18 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
SET ANSI_PADDING ON
GO
CREATE TABLE [dbo].[account] (
    [accountnumber] [varchar](50) COLLATE Chinese_PRC_CI_AS NOT NULL,
    [branchname] [varchar](50) COLLATE Chinese_PRC_CI_AS NOT NULL,
    [balance] [decimal](18, 0) NULL,
    CONSTRAINT [PK__account__07F6335A] PRIMARY KEY CLUSTERED
(
    [accountnumber] ASC
)WITH (IGNORE_DUP_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]

GO
SET ANSI_PADDING OFF
GO
USE [DBS实验案例]
GO
ALTER TABLE [dbo].[account] WITH CHECK ADD CONSTRAINT [FK_account_branch] FOREIGN KEY([branchname])
REFERENCES [dbo].[branch] ([branchname])
```

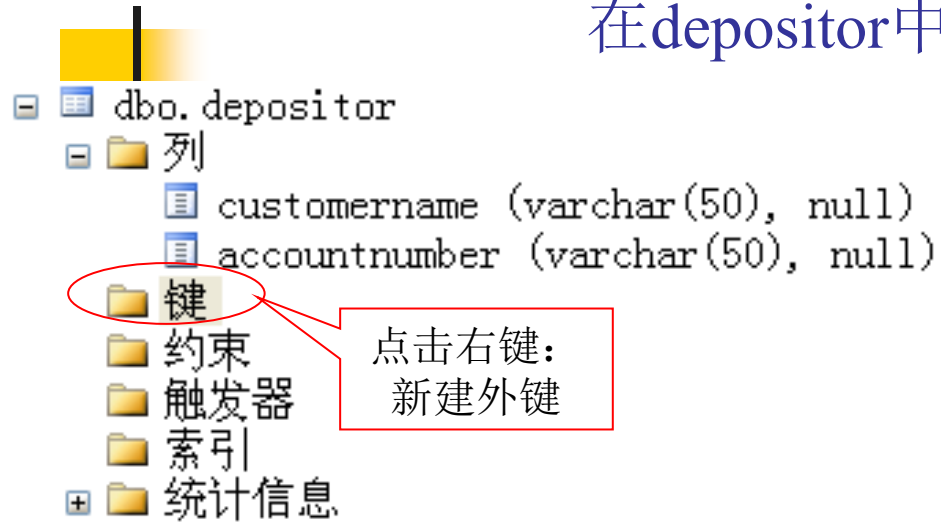
注意：

- 如何在已经创建的关系表中，加入外键定义？？
 - 在SQL Server 2005 Manager Studio下，通过图形界面，手动创建



开始时，没有定义depositor-account外键

在 depositor 中添加主键、外键



文件(F) 编辑(E) 视图(V) 项目(P) 表设计器(L) 工具(T) 窗口(W) 社区(C) 帮助(H)

新建查询(N)                 

表 - dbo.depositor* YEWEN.DBS实验...Query1.sql* 表 - dbo.branch 关系图 - YEW....Diagram_0 摘要 对象资源管

列名	数据类型	允许空
customername	varchar(50)	☒
accountnumber	varchar(50)	☒
		☐

外键关系

选定的 关系(S):

FK_depositor_depositor*

默认外键
名字

正在编辑新的 关系 的属性。 需要先填充“表和列规范”属性，然后才能接受新的 关系。

□ (常规)

□ 表和列规范

在创建或重新启用时检查现有数据 是

□ 标识

(名称)

FK_depositor_depositor

说明

□ 表设计器

□ INSERT 和 UPDATE 规范

强制外键约束

是

强制用于复制

是

点击：
创建外键

添加(A)

删除(D)

关闭(C)

列名	数据类型	允许空
customername	varchar(50)	☒
accountnumber	varchar(50)	☒
		☐

外键关系

选定的 关系(S):

FK_depositor_depositor*

正在编辑新的 关系 的属性。
后才能接受新的 关系。

常规

表 and 列规范

在创建或重新启用时检查现有

标识

(名称)

说明

表设计器

INSERT 和 UPDATE 规范

强制外键约束

强制用于复制

添加(A)

删除(D)

关闭(C)

表和列

关系名(N):

FK_depositor_account后加外键

重新定义
外键名字

主键表(P):

account

外键表:

depositor

选择被参
照关系

确定

取消



数据类型	允许空
int(50)	☐
int(50)	☒
int(50)	☐

正在编辑新的 关系 的属性。 需要先填
后才能接受新的 关系。

常规

表和列规范

在创建或重新启用时检查表是

标识

(名称)

FK_deposit

说明

表设计器

INSERT 和 UPDATE 规范

强制外键约束

是

强制用于复制

是

(D)

关闭(C)

表和列

关系名(N):

FK_depositor_account

主键表(P):

account

accountnumber

<无>

accountnumber

balance

branchname

外键表:

depositor

accountnumber

选择参照关系depositor的外键、
被参照关系account及其主键

确定

取消

Microsoft SQL Server Management Studio

文件(F) 编辑(E) 视图(V) 项目(P) 工具(T) 窗口(W) 社区(C) 帮助(H)

新建查询(N) [Icons]

关系图 - YEW....Diagram_0 摘要 对象资源管理器

连接(O) [Icons]

YEWEN (SQL Server 9.0.1399 - YEWEN\yewenbupt)

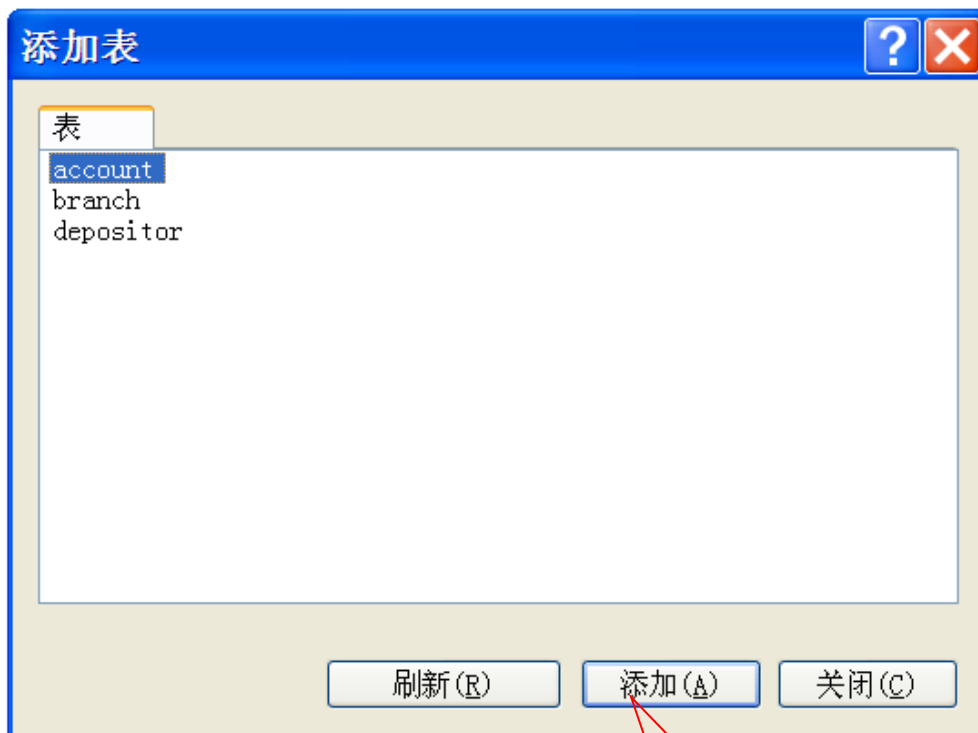
- 数据库
 - 系统数据库
 - 数据库快照
 - DBS实验案例
 - 数据库关系图
 - dbo. [Icons]
 - 表
 - 系统
 - dbo. [Icons]
 - dbo. [Icons]
 - dbo. depositor
 - 列
 - 键
 - FK_depositor_account
 - 约束
 - 触发器
 - 索引
 - 统计信息
 - 视图
 - 同义词
 - 可编程性
 - Service Broker
 - 存储
 - 安全性
 - gsmCDI_佛山
 - gsmCDI_广州
 - gsmCDI_江门
 - gsmCDIProject
 - WLANSystem广州
 - 定位1-成都
 - 定位-宁波

更新模式图

新建数据库关系图(N)

筛选器(L)

刷新(F)



依次将3张表添加到模式图，刷新

